

408 数据结构

Al-Passport 数据结构 · 数组 / 链表 / 栈 / 队列

数据结构

408 数据结构 Al-Passport 数据结构	6
"数据结构" 是什么	6
数据结构	6
"数据结构" 是什么	7
D1 数据结构 / 数据结构 ADT / 数据结构	7
1.1 数据结构	7
1.2 数据结构 ADT 是什么	7
1.3 数据结构	7
D1 是什么	8
D2 是什么 + 是什么	8
2.1 什么是 Sequential List	8
2.2 什么是 Singly Linked List	9
2.3 是什么 / 是什么	9
2.4 是什么 vs 是什么	9
2.5 是什么	9
D2 是什么	10
D3 是什么 ★ 是什么	10
3.1 是什么 Stack	10
3.2 是什么 Queue	10
★3.3 是什么	10
3.4 是什么 Deque 是什么	11
3.5 是什么	11
D3 是什么	11
D4 是什么 BF 是什么 KMP	12
4.1 是什么	12
4.2 是什么	12
4.3 ★ 是什么	12
4.4 是什么	12
D4 是什么	12
D5 是什么	12
5.1 是什么	12
5.2 ★ 是什么 + 是什么	13
5.3 是什么	13
5.4 是什么 + 是什么/是什么/是什么	13
5.5 是什么	13

5.6	平衡 BST	13
5.7	平衡 AVL	14
5.8	平衡 / 平衡 ★ WPL 和	14
5.9	堆 Heap	14
5.10	平衡	14
D5		14
D6		15
6.1	平衡	15
6.2	平衡 ★ 平衡	15
6.3	平衡	15
6.4	平衡 MST ★ 平衡	15
6.5	平衡 ★ 平衡	15
6.6	平衡 ● 平衡 "平衡"	16
6.7	平衡	16
D6		16
D7	平衡 ★ 平衡	16
7.1	平衡	16
7.2	平衡 ★ 平衡 ASL	16
7.3	平衡 / 平衡 / B 和	16
7.4	★平衡 —— 平衡	17
D7		17
D8	平衡 ★ 平衡 + 平衡/平衡	18
8.1	平衡	18
8.2	平衡	18
8.3	平衡 "平衡" 平衡	18
8.4	平衡	19
D8		19
	平衡	19
	平衡 平衡 ★ 平衡	20
C1	平衡 平衡 平衡 \$1	20
1.1	平衡	20
1.2	平衡	20
1.3	平衡 C 平衡 bin 平衡 ★ 平衡	20
1.4	平衡 "平衡 Flash 平衡 RAM" 平衡	21
1.5	平衡 平衡 16k 平衡	21
C1		21
C2	平衡 平衡 \$2/\$3 平衡	22
2.1	平衡/平衡	22
2.2	平衡 / 平衡 / 平衡 / 平衡 n 平衡 1 平衡 n+1 平衡	22
2.3	★平衡/平衡	22
2.4	平衡 Q 平衡	23
2.5	IEEE 754 平衡 ★ 平衡 + 平衡	23

O2		35
O3	★	35
3.1		35
3.2	★	35
3.3	/	36
3.4	★	36
3.5	LB	36
O3		37
O4	★	37
4.1		37
4.2	★	37
4.3	FCB	37
4.4	★	37
4.5	/	37
O4		38
O5 I/O	C7	38
5.1 I/O		38
5.2		38
5.3 SPOOLing		38
5.4		38
O5		38
[OS. ""]		38
""		40
N1		40
1.1		40
1.2 OSI vs TCP/IP	LwIP	40
1.3	+	40
N1		40
N2		41
2.1		41
2.2	★	41
2.3		41
N2		41
N3	★CRC CSMA	41
3.1		41
3.2	★CRC	41
3.3	GBN SR ★	42
3.4	★ MAC	42
3.5	VLAN	42
N3		43

N4	IP	43
4.1	IPv4	43
4.2	★CIDR	43
4.3	ARP / DHCP / ICMP	43
4.4	NAT	43
N4		44
N5	TCP	44
5.1	UDP vs TCP	44
5.2	TCP	44
5.3	★	44
5.4	★TCP	44
N5		45
N6	HTTP	45
6.1	DNS	45
6.2	★HTTP +	45
6.3		46
N6		46
	[+]	46
	+	47
◆	+ OS +	47
◆	16kHz ESP_RST_PANIC + OS	47
◆	+ =	47
	&	49
6.1	" " .	49
6.2	&	49
6.3	408 .	49
6.4		49

배열과 연결리스트

배열은 인덱스 기반의 데이터 구조로, 408번 주소부터 시작하여 1000번 주소까지 사용된다. 연결리스트는 노드가 가리키는 다음 노드의 주소를 저장하는 구조로, 1000번 주소부터 시작하여 10000번 주소까지 사용된다.

D1 배열 / 연결리스트 ADT / 문제

1.1 배열과 연결리스트

배열과 연결리스트

구분	배열	연결리스트
구조	연속된 메모리 공간	노드가 가리키는 다음 노드의 주소
탐색	O(1)	O(n)
삽입/삭제	list_add, find_next, rename	

- ① 배열과 연결리스트의 차이점
- ② 배열과 연결리스트의 사용 예시

배열은 인덱스 기반의 데이터 구조로, 408번 주소부터 시작하여 1000번 주소까지 사용된다. 연결리스트는 노드가 가리키는 다음 노드의 주소를 저장하는 구조로, 1000번 주소부터 시작하여 10000번 주소까지 사용된다.

1.2 배열 ADT 문제/문제

- 배열과 연결리스트의 차이점
- 배열과 연결리스트의 사용 예시
- ADT push/pop/insert/delete 문제
- C 배열과 연결리스트의 사용 예시

배열 ADT = "배열"을 기반으로 한 "배열"을 기반으로 한 ADT 문제

1.3 배열 문제

배열과 연결리스트의 차이점

배열 T(n)의 시간 복잡도 T(n)=3n+2는 O(n)이다. T(n)=2n²+n → O(n²) - 배열 for(i..n) for(j..n)은 O(nlogn)이다.

배열 S(n)의 시간 복잡도 S(n)=O(1)이다. in-place 배열 S(n)=O(1)이다. 배열 S(n)의 시간 복잡도 S(n)=O(n)이다.

LinkedList -> O(1) LinkedList

2.2 Singly Linked List

LinkedList

```
Node = [data | next]
      ^-Node    ^-Node (LinkedList)
```

LinkedList vs Array "LinkedList" / Array

LinkedList -> O(n) / Array -> O(1)
c new Node->next = p->next; p->next = new Node; -> O(1)
p q=p->next; p->next=q->next; free(q); -> O(1) -> O(n) p "LinkedList" O(n)

LinkedList -> tail "LinkedList" R%07u "LinkedList"

2.3 LinkedList / Array

- LinkedList [prior | data | next] O(1) LinkedList
- LinkedList next LinkedList head->next==head
- LinkedList
- LinkedList "LinkedList" O(1) "LinkedList" O(1)
- LinkedList "LinkedList" LinkedList

2.4 LinkedList vs Array

LinkedList	Array	LinkedList
LinkedList k	O(1) LinkedList	O(n) LinkedList
LinkedList	O(n) LinkedList	O(n)
LinkedList	O(n) LinkedList	O(1)
LinkedList (LinkedList)	O(1)	LinkedList O(n) LinkedList / O(1)
LinkedList	O(1)	LinkedList (LinkedList ≤ 1/2)
LinkedList	LinkedList	LinkedList
LinkedList	LinkedList	LinkedList

```
LinkedList study_task_t[256] FreeRTOS
```

2.5 LinkedList

1. LinkedList vs Array "LinkedList"
2. LinkedList "LinkedList" O(1) LinkedList trick LinkedList
3. LinkedList realloc LinkedList -> O(n) "LinkedList"

D2

Q1 n 个元素的有序数组 $a[0..n-1]$ 中， $a[i] + a[j] = n-1$ 的数对 (i, j) 有多少个？
解：设 $a[i] = x$ ，则 $a[j] = n-1-x$ 。因为数组有序，所以 $x < n-1-x$ ，即 $x < (n-1)/2$ 。因此，只需要遍历 $a[0..(n-1)/2]$ ，统计满足 $a[i] + a[n-1-i] = n-1$ 的数对即可。时间复杂度为 $O(n)$ 。

Q2 给定一个数组 $a[0..n-1]$ ，求 $\sum_{i=0}^{n-1} a[i] \cdot 2^i$ 的值。
解：直接遍历数组，计算 $a[i] \cdot 2^i$ 并累加即可。时间复杂度为 $O(n)$ 。

Q3 给定一个数组 $a[0..n-1]$ ，求 $\sum_{i=0}^{n-1} a[i] \cdot 2^i$ 的值。
解：直接遍历数组，计算 $a[i] \cdot 2^i$ 并累加即可。时间复杂度为 $O(n)$ 。

D3

3.1 Stack

栈的操作：push, pop, peek, top, empty

栈的实现：使用数组 `data`，大小为 `MaxSize`。栈顶指针 `top` 初始为 `-1`。入栈操作：`data[++top] = x`；出栈操作：`x = data[top--]`。

栈的复杂度：入栈、出栈、取栈顶元素的时间复杂度均为 $O(1)$ 。

FreeRTOS 中的栈：FreeRTOS 使用静态栈，大小由 `configTOTAL_HEAP_SIZE` 决定。可以通过 `uxTaskGetStackHighWaterMark` 获取栈的当前水位。

FreeRTOS 中的栈：FreeRTOS 使用静态栈，大小由 `configTOTAL_HEAP_SIZE` 决定。可以通过 `uxTaskGetStackHighWaterMark` 获取栈的当前水位。

3.2 Queue

队列的操作：enqueue, dequeue, front, rear

队列的实现：使用数组 `data`，大小为 `MaxSize`。队首指针 `front`，队尾指针 `rear`。入队操作：`data[rear] = x; rear = (rear+1) % MaxSize`；出队操作：`front = (front+1) % MaxSize`。

★3.3

队列的实现：使用数组 `data`，大小为 `MaxSize`。队首指针 `front`，队尾指针 `rear`。

- `front = rear = 0`
- `data[rear] = x; rear = (rear+1) % MaxSize;`
- `x = data[front]; front = (front+1) % MaxSize;`
- `front == rear`
- `(rear+1) % MaxSize == front`
- `(rear - front + MaxSize) % MaxSize` 表示队列中的元素个数
- `MaxSize - 1` 表示队列的最大容量

队列的实现：使用数组 `data`，大小为 `MaxSize`。队首指针 `front`，队尾指针 `rear`。

FreeRTOS 中的队列：FreeRTOS 使用动态队列，大小由 `configTOTAL_HEAP_SIZE` 决定。可以通过 `uxQueueGetLength` 获取队列中的元素个数。

FreeRTOS 中的队列：FreeRTOS 使用动态队列，大小由 `configTOTAL_HEAP_SIZE` 决定。可以通过 `uxQueueGetLength` 获取队列中的元素个数。

study_recorder.c

D4 字符串BF 与 KMP

4.1 字符串

- 字符串的存储
- 字符串的遍历
- 字符串的查找

4.2 字符串

- 字符串 + length
- 字符串 (\0) 与 leng (\0)

4.3 ★字符串

字符串 n 与 m

BF 字符串/字符串 - 字符串 $O(n+m)O(n \cdot m)$ aaaa...a[aa...aab]

KMP 字符串 **next** 字符串 "字符串" 字符串 $P[j] \times P[next[j]]$ 字符串 - next 字符串
"next[1]=0" 1-based $next[j] = P[0..j-1]$ 字符串 $next[next[1]=0$ 字符串
next[j] 0 nextval 字符串 **KMP** $O(m)$ **next** + $O(n)$ = $O(n+m)$ BF 字符串

字符串 next 字符串 408 字符串 "字符串" 字符串 3-5 字符串

4.4 字符串

- [parse_form[& [ssid=...&pass=... [= 字符串-字符串
- [R%07u 字符串-字符串/字符串

D4 字符串

Q1 KMP 字符串 next 字符串 S="abaabc" 字符串 P="abaabc" 字符串 next 字符串 1-based $next[1]=0$ 字符串 next[2] 字符串
P[1..1]="a" 字符串=0 $\rightarrow$ next[2]=1 字符串 2 字符串 "a" 字符串 next=1 字符串 - next[3] 字符串 P[1..2]="ab" 字符串=0 -
P[1..3]="aba" 字符串="a" 字符串 1 $\rightarrow$ next[4]=2 字符串 - next[5] 字符串 P[1..4]="abaa" 字符串="a" 字符串 1 $\rightarrow$ next[5]=2 字符串 -
P[1..5]="abaab" 字符串="ab" 字符串 2 $\rightarrow$ next[6]=3 字符串 next 字符串 $P[0..j-1]$ **Q2** KMP 字符串 BF 字符串
字符串 BF 字符串 $O(n)$ 字符串 "字符串" 字符串 $O(n \cdot m)$

KMP 字符串 next 字符串 "字符串" 字符串 "字符串" 字符串

D5 字符串字符串字符串字符串

5.1 字符串

- 字符串 = 字符串
- 字符串字符串 = 字符串
- 字符串/字符串 > 0 字符串=0 字符串

5.7 平衡 AVL

- 平衡因子 $BF \leq 1$
- $BF = \text{左子树高度} - \text{右子树高度} \in \{-1, 0, 1\}$
- 平衡操作
- LL → RR
- RR → LL
- LR → LL
- RL → RR
- AVL 树的高度 $n \leq 1.44 \log_2(n+2)$ 即 $n = O(\log n)$
- 平衡操作的时间复杂度为 $O(\log n)$

5.8 哈夫曼树 / 最优二叉树 ★ WPL 最小 + 构造

- 哈夫曼树的 WPL 最小
- $WPL(\text{哈夫曼树}) = \sum_{i=1}^n w_i l_i$
- 哈夫曼树的构造过程
- 哈夫曼树的构造过程
- 哈夫曼树的构造过程
- 哈夫曼树的构造过程
- 哈夫曼树的构造过程
- 哈夫曼树的构造过程

5.9 堆 Heap

- 堆的定义
- 堆的存储结构
- 堆的插入操作
- 堆的删除操作
- 堆的排序
- 堆的排序
- 堆的排序
- 堆的排序

5.10 图论

- 图的遍历
- 图的最短路径

D5 习题

Q1 已知 $n_0=10, n_1=5, n_2=1, n_3=1, n_4=1, n_5=1, n_6=1, n_7=1, n_8=1, n_9=1$ 求 $n_0+n_1+n_2+n_3+n_4+n_5+n_6+n_7+n_8+n_9$

Q2 已知 $ABDECF \rightarrow DBEAFC$ 求 $ABDECF \rightarrow DBEAFC$ 的变换序列

Q3 给定 $\{9, 5, 8, 2, 3\}$ 的排列 $9, 5, 8, 2, 3$ 的逆序数 $= \lfloor 5/2 \rfloor = 2 \rightarrow$ 前2个5后面有5个2, 3 (前2个3) 后面有3个5 $\rightarrow 3$ 后面有5个5 > 3 个9, 5, 8, 2, 3 $\rightarrow$ 逆序数 $\geq \lfloor n/2 \rfloor$ 的排列 $"5, 2, 9, 8, 3"$ 的逆序数为

Q4 WPL 给定 A, B, C, D 为 7, 5, 2, 4, 6, 3, 1, 2, 4 $\rightarrow$ 前6 $\rightarrow$ {7, 5, 6} - 前5, 6 $\rightarrow$ 前11 $\rightarrow$ {7, 1, 18} - WPL = $7 \times 1 + 5 \times 2 + 2 \times 3 + 4 \times 4 + 6 \times 3 = 7 \cdot 1 + 5 \cdot 2 + 2 \cdot 3 + 4 \cdot 4 + 6 \cdot 3 = 35$

Q5 给定 $n \geq 4$ 的排列 $1, 2, \dots, n-1, n$ 的逆序数 $\geq 2n-1$

D6

6.1

- $|E| = |V| - 1$
- $|E| / |V| = \text{平均度数}$
- $|E| \leq n(n-1)/2$
- $|E| \leq n^2/4$
- $|E| < |V|^2/4$

6.2

- $A[i][j] = 1$ 表示 $i \rightarrow j$ 有边 $O(n^2)$
- $O(n^2)$ 表示 $O(n^2)$
- Dijkstra 算法

6.3

- DFS 算法
- BFS 算法
- visited 数组
- DFS/BFS 算法
- DAG 算法

6.4

- n 个节点的图
- Prim 算法 $O(n^2)$
- Kruskal 算法 $O(e \cdot \log e)$
- Prim 算法
- Prim/Kruskal 算法

6.5

- Dijkstra 算法
- $n \rightarrow \log n$
- $O(n^2)$ 算法 $O((n+e) \log n)$
- Dijkstra 算法
- Floyd 算法 $O(n^3)$ $dp[k][i][j]$ 表示 $\{0..k\}$ 中 $i \rightarrow j$ 的最短路径

- B+ 索引
- 索引B树
- SPIFFS 索引 NVS 索引

7.4 ★ 索引——哈希索引

哈希 $H(key)$ 索引 $H(key) = key \% p$ p 索引 - 索引

索引 $H_i = (H(key) + i) \% m$ $[+1, +2, \dots]$ 索引 $H_i = (H(key) + i^2) \% m$ 索引 -

索引 $\alpha =$ 索引 ASL 索引 α 索引 ASL 索引 $\approx \frac{1}{2}(1 + \frac{1}{(1 - \alpha)})$ 索引 $\approx \frac{1}{2}(1 + \frac{1}{(1 - \alpha)^2})$ 索引

ASL 索引 **ASL** 索引 "索引" 索引 **ASL** 索引 $0.6(1 + \frac{1}{1 - \alpha}) + 1$ 索引
索引 / m 索引 **ASL** 索引 m 索引 n 索引 408 索引 $0..m-1$ 索引 "索引" 索引

索引 1 索引 n 索引 m 索引 2 索引 **ASL** 索引 "索引" 索引 1 索引 1 索引 **ASL** 索引

索引

索引 $H(key) = key \% 13$ $\{12, 25, 38, 51, 25, 77, 90, 5\} \dots$ 索引 - 索引 **ASL** 索引
索引 + 索引**

索引 "索引" 索引 $R0000001.FRC$ 索引 索引 $\rightarrow$ 索引 $\%$ 索引 $O(1)$ 索引 索引 NVS 索引

D7 索引

Q1 索引 **ASL** 索引 $H(key) = key \% 13$ $\{19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79\}$ 索引 **ASL** 索引
索引 $19 \% 13 = 6$ 索引 $14 \% 13 = 1$ 索引 $23 \% 13 = 10$ 索引 $1 \% 13 = 1$ $\rightarrow$ 索引 2 索引 $68 \% 13 = 3$ 索引 $20 \% 13 = 7$ 索引 $84 \% 13 = 6$ $\rightarrow$ 索引 7(20) 索引 8
 $\rightarrow$ 索引 2(1) 索引 3(68) 索引 4 索引 $55 \% 13 = 3$ $\rightarrow$ 索引 4 索引 5 索引 $11 \% 13 = 11$ 索引 $10 \% 13 = 10$ $\rightarrow$ 索引 11 索引 12 索引 $79 \% 13 = 1$ $\rightarrow$ 索引 2, 3, 4, 5, 6(19), 7(20), 8(84), 9 索引 9 索引 0, 1: 14, 2: 1, 3: 68, 4: 27, 5: 55, 6: 19, 7: 20, 8: 84, 9: 79, 10: 23, 11: 11, 12: 10
索引 **ASL** 索引 $14(1) + 1(2) + 23(1) + 68(1) + 20(1) + 84(2) + 27(4) + 55(2) + 11(1) + 10(3) + 79(9)$ 索引
 $= 1 + 2 + 1 + 1 + 1 + 2 + 4 + 2 + 1 + 3 + 9 = 27 \rightarrow ASL = 27 / 12 = 2.25$ 索引 **ASL** 索引 0~12 索引 0 $\rightarrow$ 索引 3(68), 4(27), 5(55), 6(19), 7(20), 8(84), 9(79), 10(23), 11(11), 12(10), $13 \% 13 = 0$ 索引 $\rightarrow$ 索引 14 索引 "索引" 索引
索引 "索引" 索引 "索引" 索引 **ASL** 索引

Q2 索引 $low = 100$ 索引 $mid = (100 + 150) / 2 = 125$ $\rightarrow low = 51$; $mid = (51..100) = 75 > 60 \rightarrow high = 74$;
 $mid = (51..74) = 62 > 60 \rightarrow 51..61$; $mid = (51..61) = 56 < 60 \rightarrow 57..61$; $mid = 59 < 60 \rightarrow 60$; 索引 6 索引
 $\lceil \log_2 100 \rceil + 1 = 7 \rightarrow$ 索引 $\lceil \log_2 n \rceil + 1$ 索引

Q3 索引 $\alpha =$ 索引 α 索引 1 索引 0.7 索引

$$x = (-1)^S \times (1.f)_2 \times 2^{(E-127)}$$

$$(1.f)_2 = 1 + \frac{f}{2^n}$$

IEEE754 float 32bit 1E±255 00000000 - NaN 00000000 → 0
 ∞ - ∞ 00000000 - 00000000 = (0.f)₂ × 2⁽⁻¹²⁶⁾ 00000000 (1.111...1) 254 127=127≈3.4×10³⁸
 1.0×2⁽⁻¹²⁶⁾ 1 - 000 double 64 0001 00 + 11 0000 1023 + 52 000

Speex IEEE754 Q 00000000 "000" 00000000 — 00000000 "000"

2.6 CRC

- 00000000 00000000 00000000 00000000
- Hamming 1 00000000 : 2^r ≥ n+k+1 k 00000000 n=2k 00000000 00000000 00000000 00000000
- CRC 00000000 CRC 00000000 00000000 FCS 00000000 2 NB 00
- SPIFFS CRC NVS WiFi 00 FCS 00 "00000000 FLASH/00000000" 00000000 "00" FLASH 00000000/00000000

C2

Q1 0000 -45 00000000 00101101 (8 0000) 11010010 +1 = 11010011 [-45] = 11010011 +1 0000
 Q2 0000 10011011 + 01100101 0000 10011011 = -101 01100101 = +101 00000000
 -101 ~ +101 00000000 = -101 + 101 = 0 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 110.11₂ = [1.1011 × 2² S=0] = 2 → 0000 = 2+12 10000001 f = 1011 0000 1 00000000 0 23 00
 101100...0 [0 | 10000001 | 1011000 0000 0000 0000 0000 = 0x40D80000 127 00 1—
 Q4 0000 0.5 × (-0.25) 0000 X1=0.5×32768=16384 X2=-0.25×32768=-8192 0000 = 16384×(-8192)
 = -134217728 Q30 0000 15 → -4096 0000 -4096/32= -128 0000 Q30 0000 G(x)=x³+x+1 1011 0000
 1100 00 CRC 00000000 = 00000000 11000000 11000000 ÷ 1011 01100 xor 1011 = 0111 01110 1 →
 xor 1011 = 0101 01010 0 → 01010 10100 xor 1011 = 0001010 1100010 1100010 → 00
 r 0 → 02 00000000 2 → 00000000 → 00000000

C3 §6★

3.1

00000000 → Cache SRAM → 000 DRAM → 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 Cache 00000000 00000000 00000000 00000000 PCM 0000 - -> 0000000000000000 → Ca

3.2 SRAM / DRAM / Flash +

00	SRAM 0000	DRAM 0000	Flash 0000
0000	6 0000	1 000 + 0000	00000000
00	00	0	00000000
000	00000000	00000000	000
00	CPU Cache 00000000	00000	00000000 SPI Flash

	SRAM	DRAM	Flash

DRAM refresh Flash SPIFFS text/Flash SRAM

3.3 " "

- (4 2)
- ↔ n 2^n 32 4GB
- SPI Flash** " "
- 8MB** sdkconfig [CONFIG_ESPT00LPY_FLASHSIZE_8MB=y [partitions.csv recovery [0x800000 8MB — "Flash"

nvs	0x9000	24KB	data/nvs	WiFi
phy_init	0xf000	4KB	data/phy	
factory	0x10000	3MB	app/factory	+ demo
cardid	0x356000	16KB	data/nvs	BLE Recovery
voicepack	0x35a000	480KB	data/spiffs	mp3
recordings	0x3d2000	≈3.17MB	data/spiffs	ADPCM@8kHz → 12
recovery	0x700000	1MB	app/test	BLE Recovery

= 16 0x10000=64KB 0x003D2000=3D2000_16=4006144≈3.05MB " = "

408 " " - factory recordings 3MB **cardid** " - **recovery** **BLE Recovery** **nvs** **voicepack** " **recordings** **SPIFFS** OS /

3.4 ★Cache

Cache

Tag() | Index(/) | BlockOffset()

2^b b=Cache C N " S = C/N

+

- 1 N = mod C - Index = log2C "
- 1 S **Tag Index=0** **Tag**
- = mod N S - Index = log2S - N = S - N = C

- **Write Back** Cache **Dirty** " " + " " " "
- " "
- **Write Allocate** Cache Cache " "
- **Non-Write Allocate** " "
- " " " "
- **408** **snooping** **MESI** Cache **Modified/Exclusive/Shared/Invalid** " " "
- CPU **Flash** Cache **read-only** Cache **SRAM** **Flash Cache** —

Cache / / **AMI** " "

16MB **Cache** **8KB** **16B** " "

① $A = \log_2 16\text{MB} = \log_2 2^{24} = 24$ " "

② $b = \log_2 16 = 4$ " "

③ $S = s = \log_2 (8\text{KB}/16\text{B}) = \log_2 512 = 9$ " "

④ $\text{Tag } t = A - s - b = 24 - 9 - 4 = 11$ " "

⑤ " " $2^{24}/2^4 = 2^{20}$ " " $2^{20}/2^7 = 2^{13}$ " " Tag 2^{13} " " Tag " " " "

⑥ Cache " " = $1 + \text{Tag } 11 + 16 \times 8 = 129$ **72,704 bit** **59208 KB** " "

3.6 ★ / **TLB** / **408** " "

- $\text{物理地址} \rightarrow \text{虚拟地址} = \text{VPN} \mid \text{delta}$ $\text{物理地址} = \text{PPN} \mid \text{delta}$ (")
- $\text{物理地址} \rightarrow \text{虚拟地址}$ CPU VPN PPN
- **TLB** **Cache** **TLB** **Translation-Lookaside Buffer** TLB $\rightarrow$ $O(1)$
- $\text{物理地址} \rightarrow \text{TLB}$ $\rightarrow$ PPN $\rightarrow$ OS/Flash PPN $\rightarrow$ **TLB** **TLB**
- " "
- **Flash** " "
- (P) (D) (A) " "
- **TLB** $\neq$ **TLB** " "

ESP32-C3 **MMU** $0x42000000$ / **TLB** " " + " "

C3 " "

Q1 Cache **4GB** **32** **64B** **Cache** **32KB** **64B** **Cache** $C = 32\text{KB}/64\text{B} = 512$ 2 $\rightarrow S = 512/2 = 256 \rightarrow s = \log_2 256 = 8$ **Tag** $32 - 6 - 8 = 18$ 256 **408** " "

Tag=A-b-s **Q2** **Cache** **Cache** **Q3** **AMAT** 95% 1 1 $\text{AMAT} = 0.95 \times 1 + 0.05 \times (1 + 100) = 0.95 + 5.05 = 6$ $1 + 0.05 \times 100 = 6$ **Q4** **32** **4K** $\log_2 4\text{K} = 12 \rightarrow \text{VPN} = 32 - 12 = 20 \rightarrow$ 2^{20} $2^{20} \times 4\text{B} = 4\text{MB}$ " "

C4 408 RISC-V RV32I

4.1 RISC-V RV32I 6 R I

RV32I R add/sub/and/or [31 25 24 20 19 15 14 12 11 7 6 0 funct7 | rs2 | rs1 | funct3 | rd | opcode 7 | 5 | 5 | 3 | 5 | 1] addi/lw [imm[11:0] | rs1 | funct3 | rd | opcode 12 | 5 | 3 | 5 | 1] RISC-V 7 opcode

"func add = funct7 01000000 + funct3 000_sub = funct7 01000000 + funct3 000"

addi x9, x9, -2 32-bit + 32-bit

4.2 8 +

1. addi s1, s1, 1
2. add a0, a1, a2
3. ACC
4. jal
5. lw a0, (a1) a1 RISC-V load/store "a1 +"
6. buf[i] = s0 + i*2
7. beq/blt PC PC if
8. sp/push/pop OS

" / / / " " " " " "

4.3

- beq Branch if Equal SB 13 12-bit imm = 4KB PC
- auipc + jalr 20 + 12 32-bit Flash 32-bit
- jal Jump And Link jalr jr ra jalr x0, 0(ra)

C4 408

Q1 addi x9, x9, -2 32-bit RISC-V addi 26-bit opcode = 0010011, funct3 = 000, I [imm[11:0]] [rs1] [funct3] [rd] [opcode] - rd = rs1 = x9 = 01001₂ - imm = -2 12-bit = 111111111110₂ - 111111111110 01001 000 01001 0010011 - = [0xFFF48493 (0xFFF<<20) | (9<<15) | (0<<12) | (9<<7) | 0x13] = 0xFFF00000 | 0x48000 | 0x480 | 0x13 = 0xFFF48493 ✓ - 93 84 F4 FF 12-bit

Q2 beq x1, x2, label SB 13 12-bit imm = 4KB PC auipc + jalr → " " "

Q3 CPU bootloader 0x100000000 bootloader C5

C5 5★

5.1

IF → PC 32 PC ← PC+4
ID → opcode/funct/
EX → ALU / load/store
MEM → load store SRAM
WB → ALU load

- CPI>1
- opcode+funct ALU

5.2 vs

- RISC-V/RISC
- (ROM) CISC
- =
- ESP32-C3 RISC-V RISC

5.3 ★ +

CPI=1

5 ALU throughput

n

T = 5 + (n-1) 5

n = n/T 5n 5+(n-1) → ≈ 5n

stall Read-Only SRAM RAW add x5,x1,x2 [sub x6,x5,x3 —x5 RAW WAR WAW forwarding/ bypass ALU load WB RAW stall nop load-use lw x1,0(x2) [add x3,x1,x4 load MEM nop / Speed branch penalty delay slot jal ALU

5 1000 = 5(1000+4) = 1004 = 1000/1004 ≈ 0.996

CPI 5 20% CPI 0.2×2 = 0.4 CPI ≈ 1.4 1/CP ≈ 0.71 → CPI = 1 + Σ(×)

5.4 ★/

interrupt I/O ADC exception CPU fault trap ecall abort ESP_RST_PANIC

8

- 0)
- 1)
- 2) PCPSW()
- 3) / → ISR RISC-V mepc PC
- 4) ISR
- 5) PCPSW
- 6)
- 7) RISC-V mret

- /
- ADCISR → I/O → UI
- 16k CPU ESP_RST_PANIC → "

5.5

- IVT" → ISR "
- RISC-V mtvec mepc mcause " / "

C5

Q1 510(100+1)=104 T=5+(Q2)

```
add x5, x1, x2
sub x6, x5, x3
```

sub x5 x5 forwarding add EX sub; Q3 → (fault)
OS ecall (trap) Q4 ISR SHA CPU ' / '

C6 I/O §7 ★DMA

6.1

- CPU I/O
- = × ÷8
- SPI 50MHz×8bit = 400Mbps = 50MB/s 2S 16 ×2 BCLK/LRCK
- (CPU)(CPU↔/)(I2C/SPI/UART/GPIO)
- → → (CS) SPI
-
-
- CSMA/CD I²C
- vs I2C/SPI/UART I2S

6.2 I/O 数据流图★数据流图

数据流	CPU 数据	数据	数据流
数据(数据)	数据CPU 数据	数据CPU	数据 ADC 数据
数据	数据	数据	数据 ADC
DMA	数据CPU 数据"数据"	DMA 数据数据↔数据	I2S 数据 DMA dma_desc_num=6, dma_frame_num=240

DMA 数据数据 1. CPU 数据DMA 数据数据 2. CPU 数据"数据" 3. DMA 数据数据数据数据 CPU 数据 4. 数据"数据"数据 CPU → 数据 CPU 数据数据数据 UI/WiFi 数据——数据"数据数据"数据
数据"数据" **CPU** DMA 数据数据数据 CPU 数据数据 CPU 数据/ALU → 数据 CPU 数据数据

6.3 数据

- 数据(**Buffering**)数据数据 CPU/数据数据数据 I2S DMA 数据 SPIFFS 数据 HTTP chunk 数据
- 数据(**IOP**)数据数据 **O** 数据数据 CPU 数据数据 DMA 数据

C6 数据

Q1数据数据 32 数据 50MHz = 8MB/s 50MHz /8 = 2000Mbps/8 = **250 MB/s**
Q2数据 I/O 数据数据 DMA 数据数据 CPU DMA 数据数据 CPU 数据 → 数据 UI/WiFi → DMA
Q3 DMA 数据 CPU 数据 DMA 数据 CPU 数据 DMA 数据数据

【数据数据"数据 + 数据"数据】

数据C → 数据/数据/数据 → bin → 数据 → CPU 数据 → 数据 → 数据 1. CPU 数据 = 数据 * 2¹⁸ + 1 数据 3. 数据数据 → 数据
IEEE754_(-1)^S × 1.f × 2^(E-127) 数据 127 6. Tag=A-b-s 数据 Tag AM/T=5+(n-1) 数据数据/数据

OS 108 PV 108

OS 108 PV 108 FreeRTOS OS

01 OS

1.1 OS

- OS /
- OS
- UI / / WiFi
- OS /
- OS
- OS

1.2 /

- CPU I/O
-
- RISC-V **U** **S** **M** ESP32-C3 FreeRTOS
-

1.3 system call

- OS
- `ecall` → CPU → OS → →
- `fwrite` `SPIFFS` `socket` `connect` `LwIP` `TC` `sleep` OS =
- `vs` `fprintf` `write`

1.4 OS

- OS
- `FreeRTesp_timer` → `tick` → "OS" "

01

Q1 (a) (b) (c) (d) (e) (f) (g) (h) (i) (j) (k) (l) (m) (n) (o) (p) (q) (r) (s) (t) (u) (v) (w) (x) (y) (z) (aa) (ab) (ac) (ad) (ae) (af) (ag) (ah) (ai) (aj) (ak) (al) (am) (an) (ao) (ap) (aq) (ar) (as) (at) (au) (av) (aw) (ax) (ay) (az) (ba) (bb) (bc) (bd) (be) (bf) (bg) (bh) (bi) (bj) (bk) (bl) (bm) (bn) (bo) (bp) (bq) (br) (bs) (bt) (bu) (bv) (bw) (bx) (by) (bz) (ca) (cb) (cc) (cd) (ce) (cf) (cg) (ch) (ci) (cj) (ck) (cl) (cm) (cn) (co) (cp) (cq) (cr) (cs) (ct) (cu) (cv) (cw) (cx) (cy) (cz) (da) (db) (dc) (dd) (de) (df) (dg) (dh) (di) (dj) (dk) (dl) (dm) (dn) (do) (dp) (dq) (dr) (ds) (dt) (du) (dv) (dw) (dx) (dy) (dz) (ea) (eb) (ec) (ed) (ee) (ef) (eg) (eh) (ei) (ej) (ek) (el) (em) (en) (eo) (ep) (eq) (er) (es) (et) (eu) (ev) (ew) (ex) (ey) (ez) (fa) (fb) (fc) (fd) (fe) (ff) (fg) (fh) (fi) (fj) (fk) (fl) (fm) (fn) (fo) (fp) (fq) (fr) (fs) (ft) (fu) (fv) (fw) (fx) (fy) (fz) (ga) (gb) (gc) (gd) (ge) (gf) (gg) (gh) (gi) (gj) (gk) (gl) (gm) (gn) (go) (gp) (gq) (gr) (gs) (gt) (gu) (gv) (gw) (gx) (gy) (gz) (ha) (hb) (hc) (hd) (he) (hf) (hg) (hh) (hi) (hj) (hk) (hl) (hm) (hn) (ho) (hp) (hq) (hr) (hs) (ht) (hu) (hv) (hw) (hx) (hy) (hz) (ia) (ib) (ic) (id) (ie) (if) (ig) (ih) (ii) (ij) (ik) (il) (im) (in) (io) (ip) (iq) (ir) (is) (it) (iu) (iv) (iw) (ix) (iy) (iz) (ja) (jb) (jc) (jd) (je) (jf) (jg) (jh) (ji) (jj) (jk) (jl) (jm) (jn) (jo) (jp) (jq) (jr) (js) (jt) (ju) (jv) (jw) (jx) (jy) (jz) (ka) (kb) (kc) (kd) (ke) (kf) (kg) (kh) (ki) (kj) (kk) (kl) (km) (kn) (ko) (kp) (kq) (kr) (ks) (kt) (ku) (kv) (kw) (kx) (ky) (kz) (la) (lb) (lc) (ld) (le) (lf) (lg) (lh) (li) (lj) (lk) (ll) (lm) (ln) (lo) (lp) (lq) (lr) (ls) (lt) (lu) (lv) (lw) (lx) (ly) (lz) (ma) (mb) (mc) (md) (me) (mf) (mg) (mh) (mi) (mj) (mk) (ml) (mm) (mn) (mo) (mp) (mq) (mr) (ms) (mt) (mu) (mv) (mw) (mx) (my) (mz) (na) (nb) (nc) (nd) (ne) (nf) (ng) (nh) (ni) (nj) (nk) (nl) (nm) (nn) (no) (np) (nq) (nr) (ns) (nt) (nu) (nv) (nw) (nx) (ny) (nz) (oa) (ob) (oc) (od) (oe) (of) (og) (oh) (oi) (oj) (ok) (ol) (om) (on) (oo) (op) (oq) (or) (os) (ot) (ou) (ov) (ow) (ox) (oy) (oz) (pa) (pb) (pc) (pd) (pe) (pf) (pg) (ph) (pi) (pj) (pk) (pl) (pm) (pn) (po) (pp) (pq) (pr) (ps) (pt) (pu) (pv) (pw) (px) (py) (pz) (qa) (qb) (qc) (qd) (qe) (qf) (qg) (qh) (qi) (qj) (qk) (ql) (qm) (qn) (qo) (qp) (qq) (qr) (qs) (qt) (qu) (qv) (qw) (qx) (qy) (qz) (ra) (rb) (rc) (rd) (re) (rf) (rg) (rh) (ri) (rj) (rk) (rl) (rm) (rn) (ro) (rp) (rq) (rr) (rs) (rt) (ru) (rv) (rw) (rx) (ry) (rz) (sa) (sb) (sc) (sd) (se) (sf) (sg) (sh) (si) (sj) (sk) (sl) (sm) (sn) (so) (sp) (sq) (sr) (ss) (st) (su) (sv) (sw) (sx) (sy) (sz) (ta) (tb) (tc) (td) (te) (tf) (tg) (th) (ti) (tj) (tk) (tl) (tm) (tn) (to) (tp) (tq) (tr) (ts) (tt) (tu) (tv) (tw) (tx) (ty) (tz) (ua) (ub) (uc) (ud) (ue) (uf) (ug) (uh) (ui) (uj) (uk) (ul) (um) (un) (uo) (up) (uq) (ur) (us) (ut) (uu) (uv) (uw) (ux) (uy) (uz) (va) (vb) (vc) (vd) (ve) (vf) (vg) (vh) (vi) (vj) (vk) (vl) (vm) (vn) (vo) (vp) (vq) (vr) (vs) (vt) (vu) (vv) (vw) (vx) (vy) (vz) (wa) (wb) (wc) (wd) (we) (wf) (wg) (wh) (wi) (wj) (wk) (wl) (wm) (wn) (wo) (wp) (wq) (wr) (ws) (wt) (wu) (wv) (ww) (wx) (wy) (wz) (xa) (xb) (xc) (xd) (xe) (xf) (xg) (xh) (xi) (xj) (xk) (xl) (xm) (xn) (xo) (xp) (xq) (xr) (xs) (xt) (xu) (xv) (xw) (xx) (xy) (xz) (ya) (yb) (yc) (yd) (ye) (yf) (yg) (yh) (yi) (yj) (yk) (yl) (ym) (yn) (yo) (yp) (yq) (yr) (ys) (yt) (yu) (yv) (yw) (yx) (yy) (yz) (za) (zb) (zc) (zd) (ze) (zf) (zg) (zh) (zi) (zj) (zk) (zl) (zm) (zn) (zo) (zp) (zq) (zr) (zs) (zt) (zu) (zv) (zw) (zx) (zy) (zz)

02 ★

2.1 PCB

- OS " " + OS " " "

- **OS** **PCB** **Process Control Block** **OS** **PCB** **Process Control Block** **OS** **PCB** **Process Control Block**
- **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block**

PCB **Process Control Block** **OS** **PCB** **Process Control Block** **OS** **PCB** **Process Control Block** **OS** **PCB** **Process Control Block**

FreeRTOS **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block**

2.2 **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block**

FreeRTOS **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block**

Ready **Running** **Blocked** **Ready** **Running** **Blocked** **Ready** **Running** **Blocked** **Ready** **Running** **Blocked**

- **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block**
- **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block**
- **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block**

FreeRTOS **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block**

FreeRTOS **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block** **FreeRTOS** **TCB** **Task Control Block**

2.3 **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch**

- **FreeRTOS** **PCB** **Process Control Block** **FreeRTOS** **PCB** **Process Control Block** **FreeRTOS** **PCB** **Process Control Block**
- **FreeRTOS** **PCB** **Process Control Block** **FreeRTOS** **PCB** **Process Control Block** **FreeRTOS** **PCB** **Process Control Block**

2.4 ★ **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch**

FreeRTOS **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch**

FreeRTOS **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch**

FreeRTOS **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch**

1. **FCFS** **First Come First Served** **FCFS** **First Come First Served** **FCFS** **First Come First Served**
2. **SJF** **Shortest Job First** **SJF** **Shortest Job First** **SJF** **Shortest Job First**
3. **RR** **Round Robin** **RR** **Round Robin** **RR** **Round Robin**
4. **RR** **Round Robin** **RR** **Round Robin** **RR** **Round Robin**
5. **MLFQ** **Multi-Level Feedback Queue** **MLFQ** **Multi-Level Feedback Queue** **MLFQ** **Multi-Level Feedback Queue**

SJF **Shortest Job First** **SJF** **Shortest Job First** **SJF** **Shortest Job First** **SJF** **Shortest Job First** **SJF** **Shortest Job First**

FreeRTOS **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch**

2.5 **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch**

FreeRTOS **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch** **FreeRTOS** **context switch**

Test-and-Set (TAS) / Swap Semaphore + P + V

PV P=V wait / value--; value<0 Vsignal / value++
value≤0 P value 0/V

bsp_lvgl_lock(timeout) GUI MUTEX

408

(1) K

```
empty = Kfull = 0mutex = 1  
P(empty); P(mutex); V(mutex); V(full);  
P(full); P(mutex); V(mutex); V(empty);
```

- empty/full mutex

(2) reader_count P(V)

(3) 5 ① 4 ② 5 ③

RW

```
rw = 1mutex = 1count = 0  
P(mutex); count++; if(count==1) P(rw); V(mutex); //  
...  
P(mutex); count--; if(count==0) V(rw); V(mutex); //  
P(rw); V(rw);
```

- 408 w

mutex empty mutex empty 408

PV 408 PV GUI

print() " " " " " "

2.6 ★

1 2 3 4 " "

" " " " " "

★ Max_Allocation_Need=Max-Allocation_Available Need[j]
≤ Available Available += Allocation[j]

Available = (3,3,2) P0: Allocation=(0,1,0), Max=(7,5,3) → Need=(7,4,3) P1:
Allocation=(2,0,0), Max=(3,2,2) → Need=(1,2,2) P2: Allocation=(3,0,2), Max=(9,0,2) →

Need=(6,0,0) P3: Allocation=(2,1,1), Max=(2,2,2) → Need=(0,1,1) P4: Allocation=(0,0,2), Max=(4,3,3) → Need=(4,3,1)

Available=(3,3,2) Need ≤ Available P1(1,2,2)✓ P3(0,1,1)✓ P4(4,3,1)✗ P0✗ P2✗ P3

Available=(3,3,2)+(2,1,1)=(5,4,3) P1 → (7,4,3) P4 → (7,4,5) P0 → (7,5,5) P2 → (10,5,7)

P3, P1, P4, P0, P2 Available

[illegible]

02 〇〇〇

Q1 SJF/FCFS 0 8 4 1 9 **SJF** FCFS 8 9 1 5 13 22 **40.5** **FCFS** 8 4 1 9 8 12 13 22 0 8 12 18 **25** → **Q2** FCFS FLBF →

P1: Need[1,2,2] ≤ Avail[3,3,2] → ☐☐☐☐☐☐ → Avail=[5,3,2]
 P3: Need[0,1,1] ≤ [5,3,2] → ☐☐☐ → Avail=[7,4,3]
 P0: Need[7,4,3] ≤ [7,4,3] → ☐☐☐ → Avail=[7,5,3] (P0 Alloc [0,1,0]→7? 0+0☐☐☐Allocation☐☐☐
☐☐☐☐ P1 ☐☐ Alloc[2,0,0] → ☐☐☐☐☐☐
 P4 → P2 ☐☐☐☐☐☐

→ [P1,P3,P0,P4,P2] → Need ≤ Available Available Q3

[] → "4" Q4 → "H" ↔ []

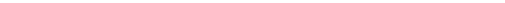
03 □□□□★□□□□□□□□□□

3.1 背景

- [illegible]

[illegible]

3.2 数据★数据



- $\mathbf{P} = \mathbf{P} \mid \mathbf{W} \Rightarrow \mathbf{W}$

- $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$ (Hit probability / Miss probability)
- $\text{Hit} \rightarrow \text{Hit} + \text{Hit} \cdot F + \text{Hit} \cdot W$
- $\text{Miss} = \text{Hit} \cdot F + \text{Miss} \cdot W$

4KB 0×1000 $0 \times 3E24$ 3 $9 - 12 = 0 \times 3E24 >> 12 = 3 \times 0 \times 3E24 \times 12 = 0 \times 9000$
 $0 \times 9000 = 0 \times 9000 + 0 \times E24 \times 9E24$ $w = \log_2$

TLB / 03.5

3.3 /

- $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$
- $\text{Hit} = \text{Hit} \cdot F + \text{Hit} \cdot W$

$\text{Hit} - \text{Hit} = \text{Hit} \cdot F - \text{Hit} \cdot W = \text{Hit} \cdot (F - W) \rightarrow \text{Hit} \rightarrow \text{Hit}$

3.4 ★

- **OPT** (Optimal) $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$
- **FIFO** (First In First Out) $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$
- **LRU** (Least Recently Used) $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$
- **CLOCK** / **CLOCK** LRU $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$

$\text{Hit} / \text{Hit} - \text{Hit} = \text{Hit} \cdot F - \text{Hit} \cdot W = \text{Hit} \cdot (F - W) \rightarrow \text{Hit} \rightarrow \text{Hit}$

$1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5$ **FIFO** $1 \rightarrow [1] (\square) 2 \rightarrow [1, 2] (\square) 3 \rightarrow [1, 2, 3] (\square) 4 \rightarrow [1, 2, 3, 4] (\square) 1 \rightarrow [2, 3, 4, 1] (\square) 2 \rightarrow [3, 4, 1, 2] (\square) 5 \rightarrow [4, 1, 2, 5] (\square) 1 (\square) 2 (\square) 3 \rightarrow [1, 2, 5, 3] (\square) 4 \rightarrow [2, 5, 3, 4] (\square) 5 \rightarrow [2, 5, 3, 4, 5] (\square)$ **LRU** $1 \rightarrow [1] (\square) 2 \rightarrow [1, 2] (\square) 3 \rightarrow [1, 2, 3] (\square) 4 \rightarrow [1, 2, 3, 4] (\square) 1 \rightarrow [2, 3, 4] (\square) 2 \rightarrow [3, 4] (\square) 5 \rightarrow [3, 4, 5] (\square) 1 \rightarrow [4, 5] (\square) 2 \rightarrow [5] (\square) 3 \rightarrow [\square] (\square)$ **OPT** $1 \rightarrow [1] (\square) 2 \rightarrow [1, 2] (\square) 3 \rightarrow [1, 2, 3] (\square) 4 \rightarrow [1, 2, 3, 4] (\square) 1 \rightarrow [2, 3, 4] (\square) 2 \rightarrow [3, 4] (\square) 5 \rightarrow [3, 4, 5] (\square) 1 \rightarrow [4, 5] (\square) 2 \rightarrow [5] (\square) 3 \rightarrow [\square] (\square)$

LRU $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$ $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$

CLOCK $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$ $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$

3.5 TLB

- **thrashing** (Thrashing) $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$
- **TLB** (Translation Lookaside Buffer) $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$
- **P1** | **P2** $\text{Hit} = \text{Hit} \cdot F + \text{Miss} \cdot W$

O4

Q1 1KB 4B inode 10 + 1 1024/4 = 256 1KB = 256KB 10 = 10KB + 1 256KB + 256² × 1KB ≈ 64MB 3000 10 256 → 3030 4096 = 4096 bit = 512 512 100 55, 58, 39, 18, 90, 150, 38, 180 SSTF 90 10 → 55 (35)? 90 55 35 58 37 55 → 58 (3) → 39 (19) → 38 (1) → 18 (20) → 150 → 160 → 180 SSTF.

Q4 "filename" rename —

O5 I/O C7

5.1 I/O

```
os_sys_open/read/write
os_sys_

```

- bsp_* API →
- DMA** / C7 "CPU "

5.2

- /
- /
- I2S + DMA SPIFFS HTTP — " CPU "

5.3 SPOOLing

- " " " " SPOOLing
- " " SPOOLing

5.4

- / / SPOOLing
- /

O5

Q1 /Flash **C7** **Q5** **Q2** SPOOLing

[OS ""]

- SJF RR
- PV**- 4

3. $\frac{1}{n} \sum_{i=1}^n \frac{1}{f_i} + \frac{1}{n} \sum_{i=1}^n \frac{1}{f_i^2}$
 4. $\frac{1}{n} \sum_{i=1}^n \frac{1}{f_i} = \frac{1}{n} \times \sum_{i=1}^n \frac{1}{f_i} + \frac{1}{n}$
 5. FIFO/LRU/OPT/CLOCK Belady FIFO
 6. $\frac{1}{n} \sum_{i=1}^n \text{inode}_i + \frac{1}{n} \sum_{i=1}^n \text{SSTF/SCAN}$
-

1. 网络基础

1.1 网络基础

N1 网络基础

1.1 网络基础

- 网络基础
- 网络基础

1.2 OSI 模型 vs TCP/IP 模型

OSI 7层	TCP/IP 5层	应用层
应用层	应用层	HTTP, DNS, FTP, SMTP, DHCP
表示层	表示层	TCP, UDP
会话层	会话层	IP, ARP, ICMP
传输层	传输层	WiFi (802.11), MAC
网络层	网络层	网络层
数据链路层	数据链路层	数据链路层
物理层	物理层	物理层

- 网络基础

1.3 网络基础

- 带宽 bandwidth 单位 bps bit/s
- 网络基础
- 网络基础
- 网络基础
- 网络基础
- 网络基础
- 网络基础
- 网络基础

网络基础

N1 网络基础

Q1 1000B 10Mbps 2000km $2 \times 10^3 \times 10^6 \times 8 \text{ bit} / 10^6 \text{ bps} = 8000 / 10^8 \text{ ms} = 8 \times 10^{-5} \text{ s}$
 $2000 \times 10^3 / 2 \times 10^8 = 10^{-2} \text{ s} = 10 \text{ ms}$

Q2 网络基础

N2 問題

2.1 問題

- 0-1 変換: 0 → 1, 1 → 0
- 0-1 変換: 0 → 1, 1 → 0
- **ASK FSK QAM** 変換: 0 → 1, 1 → 0
- WiFi **OFDM** 変換: 0 → 1, 1 → 0

2.2 問題★問題/問題

- 問題 **FDM** 変換: 0 → 1, 1 → 0
- 問題 **TDM** 変換: 0 → 1, 1 → 0
- 問題 **WDM** 変換: 0 → 1, 1 → 0
- 問題 **CDM** 変換: 0 → 1, 1 → 0

CDMA 変換: $a = (1, -1, 1, -1)$, $b = (-1, 1, -1, 1)$
 $a \cdot b = 1 \cdot (-1) + (-1) \cdot 1 + 1 \cdot (-1) + (-1) \cdot 1 = -1 - 1 - 1 - 1 = -4 \neq 0 \rightarrow$ 問題 変換: 0 → 1, 1 → 0

2.3 問題問題問題

- 問題問題問題問題問題
- 問題問題 $C = 2W \log_2 V$ $W =$ 問題 $V =$ 問題/問題
- 問題問題 $C = W \log_2 (1 + S/N)$ $S/N = 10^{(問題dB/10)}$
- 問題問題問題問題"問題/問題/問題"問題 bps 問題

問題問題 3kHz 問題 $S/N = 10^3 = 1000$ $C = 3000 \times \log_2(1 + 1000) \approx 3000 \times 9.97 \approx 30\text{kbps}$

N2 問題

Q1 問題問題 4kHz 問題 $S/N = 10^2 = 100$ $C = 4000 \times \log_2(101) \approx 4000 \times 6.66 \approx 26.6\text{kbps}$
Q2 Nyquist 問題 3kHz 問題 $V = 2 \times 3000 = 6000$ 問題/s 問題問題 = $6000 \times \log_2 8 = 6000 \times 3 = 18\text{kbps}$
問題問題問題

N3 問題★CRC 問題 CSMA 問題

3.1 問題問題問題

- $P =$ 問題 + 問題問題 + 問題 FCS 問題
- 問題問題問題問題/ 問題 / 問題問題
- **MAC** 問題 48 問題/ 問題問題問題 WiFi MAC $\rightarrow$ 問題 SSID 問題 4 問題

3.2 問題★CRC 問題

- 問題問題 1 問題問題問題問題
- 問題問題 1 問題問題 C2 問題

- **CRC** (Cyclic Redundancy Check) FCS (Frame Check Sequence) 2 바이트 C2 Q5 바이트 FRC (Frame Reliability Check) / SPIFFS / CRC

3.3 GBN/SR ★

RTT = 2τ

- ACK-ACK → ACK = (ACK + 1) / (ACK + 1) ACK
- ACK/ACK
- **GBN** / **Go-Back-N** > 1 ACK ≤ 2ⁿ - 1 n =
- **SR** / **Selective Repeat** ACK ≤ 2ⁿ - 1
- **TCP** "ACK + ACK + ACK / ACK" N5

1ms 1000B 10Mbps 1000 × 8 / 10M = 0.8ms U = 0.8 / (0.8 + 2 × 1) = 0.8 / 2.8 ≈ **28.6%** N U = N × (ACK) / (ACK + 2τ) TCP

GBN/SR N U = N × T / (T + 2τ) N ≥ (2τ + T) / T N ≥ 2τ / T + 1 N GBN ≤ 2ⁿ - 1 SR ≤ 2ⁿ - 1 n = 3 408

1000B 1Mbps 25ms 1000 × 8 / 10⁶ = 8ms RTT = 2τ = 50ms U = 8 / (8 + 50) = 8 / 58 ≈ **13.8%** ② GBN ≥ 90% N × 8 / (8 + 50) ≥ 0.9 → N × 8 ≥ 0.9 × 58 = 52.2 → N ≥ 6.525 → **N = 7** ③ 2ⁿ - 1 ≥ 7 N 2ⁿ ≥ 8 → n = 3 7 2ⁿ - 1 ≥ 7 R 2ⁿ ≥ 14 → n = 4 2³ = 8 ≥ 7

2ⁿ GBN "ACK" "ACK" < ACK ≤ 2ⁿ - 1 SR

3.4 ★ MAC

- ALOHA/CSMA/CSMA-CD FDM/TDM N2
- **CSMA**
- 1 CSMA
- CSMA
- p CSMA
- **CSMA/CD**
- CSMA/CA 802.11 WiFi DIFS (Priority) → ACK RTS/CTS
- **ALOHA**

CSMA/CD 2500m 100Mbps 25 × 10⁻⁶ × 10⁸ = 12.5μs 2τ = 25μs 100 × 10⁶ × 25 × 10⁻⁶ = 2500 bit ≈ **312.5B**

3.5 VLAN

- 802.3 + DA + SA + + FCS MAC 48
- **VLAN** 802.1Q 4 Tag VLAN ID

N4

Q1 200.100.20.130/27 255.255.255.224 256-224=32 130/32=4...2 → 0+4×32=200.100.20.128 128+32-200.100.20.159 32-30 Q2 192.168.5.0/24 3 3 → 3 = 2³=8 8-3=5 → Q3 ARP 2PH2 30B ARP ARP MAC → MAC

N5 TCP

5.1 UDP vs TCP

	UDP	TCP
	8B	20B+
	DNS(53)/SNTP(123)/DHCP/	HTTP/FTP/SMTP
	SNTP DHCP	HTTP /

5.2 TCP

- seq ack SYN/ACK/FIN/RST/PSH/URG
- SYN(seq=x) → SYN+ACK(seq=y, ack=x+1) → ACK(ack=y+1) → HTTP
- FIN → ACK → FIN → ACK → TIME_WAIT 2MSL

5.3

- + " n-1" ack=n
- ack=n " n" 0~n-1
- ACK rwnd

1000B seq=1000 ack=2000 1999 1000 seq=2000 =+1"

5.4 TCP

cwnd ssthresh Slow Start cwnd 1 MSS RTT cwnd≥ssthresh RTT +1 AIMD ssthresh=cwnd/2 cwnd=1 ACK ssthresh=cwnd/2 cwnd=ssthresh 1 = AIMD rwnd cwnd min(cwnd, rwnd) 408

cwnd $\text{ssthresh}=16$ $\text{cwnd}=1,2,4,8,16 \rightarrow 16$ $\text{cwnd}=17,18,19,20$ $\text{cwnd}=20$ $\text{ssthresh}=20/2=10$ $\text{cwnd}=1$ $\text{cwnd}=1,2,4,8 \rightarrow 10$ $\text{ssthresh}=10$ $\text{ssthresh}=20/2=10$ $\text{cwnd}=10$ $\text{cwnd}=1$ $\text{cwnd}=\text{ssthresh}$

ssthresh $\text{cwnd}=34$ $\text{ssthresh}=\text{cwnd}/2=17$ $\text{cwnd}=\text{ssthresh}=17$ - ② RTT $\text{cwnd}+1$ $18,19,20,\dots$ $\text{ssthresh}=\text{cwnd}/2=12$ $\text{cwnd}=1$ $1,2,4,8,12$ RTT cwnd $\text{cwnd}=34$ ssthresh

cwnd $\text{cwnd} \times 2 \text{ per RTT} = \dots$

N5

Q1 TCP 3 ACK $\text{ssthresh}=\text{cwnd}/2$ $\text{cwnd}=\text{ssthresh}$ 4082

TCP SYN-ACK Q3 SYN-ACK Q4

MSS/RTT TCP $\text{MSS}=1000\text{B}$ $\text{RTT}=10\text{ms}$ $\text{cwnd}=20$ $\text{MSS}=2000\text{B}$ $\text{RTT}=0.01\text{s}$

$\rightarrow = 20000/0.01 = 2 \text{ MB/s} = 2 \times 10^6 \text{ B/s}$ $\approx \text{cwnd} \times \text{MSS} / \text{RTT}$ $\text{cwnd}=5000\text{B}$ cwnd $\text{min}(\text{cwnd}, \text{rwnd}) = 5000\text{B} \rightarrow 5000/1000 = 5$ rwnd

N6 HTTP 网络/网络

6.1 DNS

- $\text{www} \rightarrow \text{www.com/.cn} \rightarrow \text{www.baidu.com} \rightarrow \text{www}$
- www
- A IPv4 AAAA IPv6 CNAME MX NS
- UDP:53** TCP
- $192.168.4.1$ DNS SNTP

6.2 HTTP 网络 + 网络

- TCP** $\text{STUDY_WIFI_AP_PORT}=80$
- URL URL URL URL
- $\text{GET /rec/dl?seq=7 HTTP/1.1 ?seq=7}$ **query**
- GET
- POST** POST /save SSID
- $\text{PUT/DELETE/HEAD/OPTIONS}$
- 1xx 200 OK 3xx 404 Not Found 5xx
- $\text{Host Content-Type application/json audio/wav Content-Length Content-Disposition: attachment; filename=... Transfer-Encoding: chunked}$
- TCP HTTP/1.0

- 保持 **keep-alive** 保持 TCP 连接 HTTP/1.1 支持持久连接
- 配置 N6.2 使用 **SoftAP** 模式 SSID= 默认 Mhttp://192.168.4.1 2) GET / 或 HTML POST / 保存 [ssid=...&pass=... 配置] → 配置 NVS → 配置 AP → **HTTP Server** 1) GET /rec/dl?seq=N → 配置 FRC → 配置 WAV → 配置 **chunked** 配置 R0000007.wav

6.3 网络配置

- **FTP** 配置 21 + 配置 20 配置
- 配置 SMTP 25/465 配置 POP3 110 配置 IMAP 143 配置 MIME 配置/配置
- **SNMP** 配置 161/162 配置
- **DHCP** 配置 N4.3 配置

N6 配置

Q1 配置 /rec/dl?seq=999 配置 **404 Not Found** 4xx **Q2** 配置 www.baidu.com 配置 配置 .com 配置 baidu.com 配置 "配置 .com" **Q3** 配置 Content-Disposition: attachment; filename=R0000007.wav 配置 **Q4** chunked Transfer-Encoding: chunked 配置 chunked 配置 chunked 配置 → 配置 chunked

【配置 "配置 + 配置 "配置】

配置 → 配置 → 配置 = 配置 $C = W \log_2(1 + S/N)$ $C = 2W \log_2 V$ 3. CSMA/CD 配置 $\geq \frac{1}{2} \times 2\tau$ 配置 / CDMA 配置 = 256 - 配置 / 配置 TCP cwnd 配置

ESP8266 + 物联网应用

物联网应用 = 硬件 + 软件 + 网络 + 应用

◆ 物联网应用 = 硬件 + OS + 应用

物联网应用

- 1. 硬件/软件 **ADC** 物联网应用 **WiFi** 物联网应用 **bsp_button.c** 物联网应用 **bsp_wifi.c** 物联网应用
- 2. 硬件 **I/O** 物联网应用 **ADC** **ISR** 物联网应用 **poll_evt** 物联网应用 **DMA** 物联网应用
- 3. 物联网应用 **OS** 物联网应用 **bsp_lvgl_lock(timeout)** 物联网应用 + 物联网应用 **OS** 物联网应用 **02.5** 物联网应用
- 4. 硬件/软件-物联网应用 **DS + OS** 物联网应用 **DS D3** 物联网应用 **poll_evt** 物联网应用 **OS** 物联网应用 + 物联网应用
- 5. **CPU** 物联网应用 **switch/if** 物联网应用 **beq** 物联网应用 **jal** + 物联网应用 **C4/C5** 物联网应用
- 6. 物联网应用 + 物联网应用 **250ms** 物联网应用 **250ms** 物联网应用 **250ms** 物联网应用

物联网应用 = ADC(I/O) → DS(DS) → UI(OS) → CPU(CPU) 物联网应用

◆ 物联网应用 16kHz ESP_RST_PANIC 物联网应用 + OS

物联网应用

- 物联网应用 **C1/C3** 物联网应用 **Flash** 物联网应用 **RAM** 物联网应用
- **OS O3** 物联网应用 **ESP** 物联网应用 + 物联网应用 **33KB** 物联网应用 **16KB** 物联网应用 **16KB** 物联网应用 → 物联网应用
- 物联网应用 **C5** 物联网应用 **malloc/xTaskCreate** 物联网应用 **abort** 物联网应用 → **IDF** 物联网应用 **_reset_reason=PANIC** 物联网应用
- 物联网应用 **C3** 物联网应用 **Speex WB** 物联网应用 **8kNB** 物联网应用
- **OS** 物联网应用 **uxTaskGetStackHighWaterMark()** 物联网应用 **HWM=4294967295** 物联网应用 **0xFFFFFFFF** 物联网应用 **HWM** 物联网应用

物联网应用① 物联网应用 **Flash** 物联网应用 + **SRAM** 物联网应用② 物联网应用/物联网应用③ 物联网应用 + 物联网应用 → 物联网应用 → **abort** 物联网应用 → 物联网应用④ 物联网应用

◆ 物联网应用 + 物联网应用 = 物联网应用"物联网应用"

物联网应用 **N1~N6** 物联网应用 1. **SoftAP** 物联网应用 **192.168.4.1** 物联网应用 → **N4.1/NAT** 物联网应用 **WCSMA/CA** 物联网应用 **ACK** 物联网应用 **N3.4** 物联网应用 **GET http://192.168.4.1/** 物联网应用 → **HTTP Server** 物联网应用 **TCP:80** 物联网应用 **HTTP** 物联网应用 **TCP** 物联网应用 → 物联网应用/物联网应用 → **HTTP/1.1 keep-alive** 物联网应用 **N5.2** 物联网应用 **N6.2** 物联网应用 3. 物联网应用 **POST /save** 物联网应用 **N6.2 ssid=..&pass=..** 物联网应用/物联网应用 **parse_form** 物联网应用 **DS D4** 物联网应用 → **NVS** 物联网应用 **OTA** 物联网应用 **DHCP** 物联网应用 **Discover/Offer/Request/Ack** 物联网应用

4 IP GET /rec/dl?seq=N → FRC → **404 Not Found** Content-Disposition →
R...wav 6.2 6. TCP N5.3 +ACK+

MAC IP/TCP' DNS/DHCP

